



Full Step-by-Step Guide: Hadoop MapReduce in Java on Ubuntu



Prerequisites



Ensure the following are already installed and configured:

- **Java JDK**
- **Hadoop (e.g., 3.4.0)** properly configured under \$HADOOP_HOME
- A system user like hduser_



Create Project Directory and Java Package



Where: Terminal as hduser_@guna

```
mkdir ~/MapReduceTutorial
```

```
cd ~/MapReduceTutorial
```

```
mkdir SalesCountry
```



Create Java Files



Where: Inside ~/MapReduceTutorial/SalesCountry

a. SalesMapper.java

```
package SalesCountry;
```

```
import java.io.IOException;
```

```
import org.apache.hadoop.io.*;
```

```
import org.apache.hadoop.mapred.*;
```

```
public class SalesMapper extends MapReduceBase implements Mapper<LongWritable, Text, Text, IntWritable> {
```

```
    private final static IntWritable one = new IntWritable(1);
```

```
    public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output, Reporter reporter) throws IOException {
```

```
        String[] fields = value.toString().split(",");
```

```
        if (fields.length > 7) {
```

```
            output.collect(new Text(fields[7]), one);
```

```
    }  
    }  
}
```

b. SalesCountryReducer.java

```
package SalesCountry;  
  
import java.io.IOException;  
import java.util.Iterator;  
import org.apache.hadoop.io.*;  
import org.apache.hadoop.mapred.*;  
  
public class SalesCountryReducer extends MapReduceBase implements Reducer<Text, IntWritable,  
Text, IntWritable> {  
    public void reduce(Text key, Iterator<IntWritable> values, OutputCollector<Text, IntWritable>  
output, Reporter reporter) throws IOException {  
        int total = 0;  
        while (values.hasNext()) {  
            total += values.next().get();  
        }  
        output.collect(key, new IntWritable(total));  
    }  
}
```

c. SalesCountryDriver.java

```
package SalesCountry;  
  
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.*;  
import org.apache.hadoop.mapred.*;  
  
public class SalesCountryDriver {  
    public static void main(String[] args) {  
        JobClient client = new JobClient();
```

```
JobConf conf = new JobConf(SalesCountryDriver.class);

conf.setJobName("SalePerCountry");
conf.setOutputKeyClass(Text.class);
conf.setOutputValueClass(IntWritable.class);
conf.setMapperClass(SalesMapper.class);
conf.setReducerClass(SalesCountryReducer.class);
conf.setInputFormat(TextInputFormat.class);
conf.setOutputFormat(TextOutputFormat.class);
FileInputFormat.setInputPaths(conf, new Path(args[0]));
FileOutputFormat.setOutputPath(conf, new Path(args[1]));

client.setConf(conf);
try {
    JobClient.runJob(conf);
} catch (Exception e) {
    e.printStackTrace();
}
}
```

Compile the Java Files

 **Where: Inside ~/MapReduceTutorial**

```
cd ~/MapReduceTutorial
```

```
javac -d . SalesCountry/SalesMapper.java SalesCountry/SalesCountryReducer.java
SalesCountry/SalesCountryDriver.java
```

✓ After this, .class files will appear inside the SalesCountry/ folder.

Create Manifest.txt

 **Where: Inside ~/MapReduceTutorial**

```
gedit Manifest.txt
```

 Paste:

Main-Class: SalesCountry.SalesCountryDriver

 Ensure **you press ENTER** after this line before saving and closing the file.

5 Package into a JAR

 **Where: Still inside ~/MapReduceTutorial**

```
jar cfm ProductSalePerCountry.jar Manifest.txt SalesCountry/*.class
```

 Confirm:

```
jar tf ProductSalePerCountry.jar
```

6 Download the Dataset

 **Any browser:**

Download CSV from:

 <https://drive.google.com/uc?export=download&id=1tP8AJGSgDXwI12r2Ap07GyamMj1o0iDD>

Then, move it to:

```
mkdir ~/inputMapReduce
```

```
mv ~/Downloads/SalesJan2009.csv ~/inputMapReduce/
```

7 Start Hadoop Services

 **Terminal as hduser_@guna**

```
$HADOOP_HOME/bin/hdfs namenode -format    # (only once)
```

```
$HADOOP_HOME/sbin/start-dfs.sh
```

```
$HADOOP_HOME/sbin/start-yarn.sh
```

```
jps # Confirm: should show NameNode, DataNode, ResourceManager, NodeManager
```

8 Upload Dataset to HDFS

 **Terminal**

```
hdfs dfs -mkdir /inputMapReduce
```

```
hdfs dfs -copyFromLocal ~/inputMapReduce/SalesJan2009.csv /inputMapReduce/
```

```
hdfs dfs -ls /inputMapReduce
```

9 Run the MapReduce Job

✅ If `/mapreduce_output_sales` does not exist:

```
hadoop jar ProductSalePerCountry.jar /inputMapReduce /mapreduce_output_sales
```

❌ If it already exists (Error: `FileAlreadyExistsException`):

```
hdfs dfs -rm -r /mapreduce_output_sales
```

```
hadoop jar ProductSalePerCountry.jar /inputMapReduce /mapreduce_output_sales
```

10 View the Output

📌 Terminal

```
hdfs dfs -cat /mapreduce_output_sales/part-00000
```

✔ You'll see output like:

```
United States 120
```

```
United Kingdom 83
```

```
India 56
```

📖 Optional: View via Web UI

- **HDFS Web UI:** <http://localhost:9870>
- **YARN ResourceManager:** <http://localhost:8088>

Navigate to **Browse the FileSystem** → `/mapreduce_output_sales/` → `part-00000`

✅ You're Done!

Your Java MapReduce job is now fully running on Hadoop and counting product sales by country. Let me know if you'd like to:

- Automate this entire process with a build+run script
- Add logs or enhancements to your Mapper/Reducer
- Learn how to use the newer `org.apache.hadoop.mapreduce` API

Let me know how you'd like to proceed!